
CAST 수식

식 (1)~(15) · Word 수식 개체로 넣을 소스

쓰는 법

Word 2016 이상: 삽입 → 수식 → (수식 도구) LaTeX 선택 → 아래 한 줄을 붙여넣고 변환
각 식의 ... 안쪽만 복사한다. % 로 시작하는 줄은 설명이니 복사하지 않는다.

왜 LaTeX 인가

현재 CAST.docx 의 수식 12개가 전부 일반 텍스트다 (Word 수식 개체 0개).

그래서 수식처럼 보이지 않는다. LaTeX 로 넣으면 학술지 조판과 같은 모양이 된다.

주의

Word 의 LaTeX 모드는 일부 명령을 지원하지 않는다.

`\mathit`, `\text`, `\operatorname` 등이 안 먹으면 아래 [Word 대안] 을 쓴다.

변환 후에는 반드시 눈으로 확인할 것.

기호 규약은 공식.txt [0] 을 따른다. 특히

미배정 변수는 z_j (기존 s_j 는 제출 시각과 충돌하므로 바꿨다)

예측은 발행 시각을 뒤편자로 명시: $\hat{C}_r^{\tau}(t)$

1. 이미 논문에 있는 식 — 텍스트를 수식 개체로 바꾸기만 하면 된다

(1) 목적함수 — 총 배출량 최소화 문단 446

$$\min \sum_{j \in J} P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t) dt \quad (1)$$

(2) 모든 작업은 정확히 한 리전에 배정 문단 448

$$\sum_{r \in R} x_{jr} = 1, \quad \forall j \in J \quad (2)$$

(3) 용량 제약 — 이 식의 `\forall t` 가 연속 시각이다 문단 449

2026-09-16 구현 버그를 판정한 근거가 이 식이었다

$$|\{j : r(j) = r, \tau_j \leq t < \tau_j + d_j\}| \leq \lfloor \eta \cdot cap_r \rfloor, \quad \forall r \in R, \forall t \quad (3)$$

(4) 실행은 제출 이후이고 마감 이내 문단 450

$$s_j \leq \tau_j, \quad \tau_j + d_j \leq D_j, \quad \forall j \in J \quad (4)$$

(5) 시간 지연과 네트워크 지연의 합이 예산 이내 문단 451

본문이 밝힌 대로 구현에서 강제되지 않고 수치상 구속되지도 않는다

$$(\tau_j - s_j) + \ell_{o_j r(j)} \leq L_j, \quad \forall j \in J \quad (5)$$

(6) 정규화 — 탄소는 슬롯별, 지연은 고정 분모 문단 475

$$\tilde{M}_r = \frac{\hat{C}_r}{\max_{r'} \hat{C}_{r'}}, \quad \tilde{\ell}_{or} = \frac{\ell_{or}}{244} \quad (6)$$

(7) 슬롯 ILP 목적함수 문단 479

$$\left| \min \sum_j \sum_r \left(\alpha \tilde{M}_r + (1 - \alpha) \tilde{\ell}_{o_j r} \right) x_{jr} + P_{slack} \sum_j z_j \right. \quad (7)$$

(8) 슬롯 ILP 제약 문단 480

$$\left| \sum_r x_{jr} + z_j = 1 \quad \forall j, \quad \sum_j x_{jr} \leq a_r \quad \forall r \right. \quad (8)$$

(9) 파레토 무류점 – 기여 1 문단 484

$$\left| \alpha^* = \arg \min_{\alpha} \sqrt{\tilde{\lambda}(\alpha)^2 + \tilde{\mu}(\alpha)^2} \right. \quad (9)$$

(10) 탄소 비용과 지연 비용의 0-1 정규화 문단 499

$$\left| \hat{C}(t) = \frac{\bar{C}(t) - C_{\min}}{C_{\max} - C_{\min}}, \quad \hat{D}(t) = \frac{t - t_{\text{early}}}{t_{\text{late}} - t_{\text{early}}} \right. \quad (10)$$

(11) 슬롯 점수 – 시간 이동의 판단 기준 문단 500

$$\left| score(t) = \alpha(k) \hat{C}(t) + (1 - \alpha(k)) \hat{D}(t), \quad \alpha(k) = \frac{6-k}{5} \right. \quad (11)$$

(12) 탄소 회계 – 배출량은 실측값으로 산정 문단 514

** 이미 있는 식이다. 새 식은 (13) 부터다. **

$$\left| E_j = P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t) dt \right. \quad (12)$$

2. 새 식 – 용량 인지 온라인 시간 이동 (§3.6 신설)

(13) 가용 자리 – 식 (3) 을 가용 자리 형태로 다시 쓴 것

논문에 이렇게 쓸 것:

"5.2절에서 식 (3) 으로 선언했으나 구현하지 않았던 제약을

이 알고리즘이 실제로 강제한다."

$$\left| n_r(t) = |\{j : r(j) = r, \tau_j \leq t < \tau_j + d_j\}|, \quad a_r(t) = \lfloor \eta \cdot cap_r \rfloor - n_r(t) \right. \quad (13)$$

(14) 희망 집합 – 남은 윈도우에서 지금이 최선인 작업

score 는 식 (11) 을 그대로 쓴다. 새 점수 함수를 만들지 않는다.

단 예측은 발행 시각이 t 인 것을 쓴다 (규약 R4).

$$\left| Q(t) = \{j : s_j \leq t, \tau_j \text{ 미정} \} \right. \quad (14)$$

$$\left| P_r(t) = \left\{ j \in Q(t) : r(j) = r, \arg \min_{t' \in W_j(t)} score_j(t') = \max(t, s_j) \right\} \right.$$

(15) 우선순위와 선택 – 순수 EDF

탄소 이득으로 정렬하면 나중에 측정할 지표로 승자를 미리 뽑는 셈이라

순환 논리가 된다. 여유만 보면 그 문제가 없고, 여유 0 이 자동으로 1순위다.

$$\left| u_j(t) = (D_j - d_j) - t \right. \quad (15)$$

$$F_r(t) = \{j \in Q(t) : r(j) = r, u_j(t) < 1\}$$

$$S_r(t) = F_r(t) \cup R_r(t) [1 \dots \max(a_r(t) - |F_r(t)|, 0)]$$

R_r(t) 는 P_r(t) \setminus F_r(t) 를 u_j(t) 오름차순으로 정렬한 것

3. 규약－번호는 안 붙이되 본문에 반드시 적을 것

2026-09-16 구현 버그가 전부 여기서 나왔다 (공식.txt [2] 참고)

R1 후보 슬롯 격자－벽시계 정시에 현재 시점 자체를 더한다

첫 항이 "지금 당장 실행" 이다. 이게 없으면 제출 시각이 정시가 아닌 작업이

다음 정시까지 밀리고, 지연 예산이 1시간 미만인 작업은 마감율 여긴다.

$$W_j(t) = \{\max(t, s_j)\} \cup \{t' \in \mathbb{Z} : \lceil \max(t, s_j) \rceil \leq t' \leq \min(D_j - d_j, t + H)\}$$
R1

R4 예측 발행 시각－두 알고리즘의 차이가 정확히 여기다

사전 예약형은 제출 시점 예측 하나로 전 윈도우를 평가하고

롤링형은 평가하는 매 슬롯의 최신 예측으로 다시 평가한다

$$\bar{C}_j^{(\tau)}(t') = \frac{1}{d_j} \int_{t'}^{t'+d_j} \hat{C}_{r(j)}^{(\tau)}(u) du$$
R4

4. Word 대안－LaTeX 모드가 명령을 못 알아들을 때

Word 수식 편집기는 UnicodeMath(선형 서식)도 받는다.

수식 안에서 아래처럼 치고 스페이스를 누르면 변환된다.

`\lfloor` -> `[ \rfloor` -> `]`

`\tau` -> `\tau` `\alpha` -> `\alpha`

`\hat{C}` -> C 위에 모자 (C\hat 다음 스페이스)

`\sum_` -> $\sum$ 아래첨자

`\le` -> $\leq$ `\forall` -> $\forall$

`a_r(t)` -> 아래첨자 r (a_r 다음 스페이스)

지금 본문이 쓰는 τ_j 같은 표기가 이미 UnicodeMath 와 비슷하다.

수식 개체 안에 넣고 스페이스만 누르면 상당수가 그대로 변환된다.

5. 남은 작업

[] 기존 식 (1)~(12) 를 전부 Word 수식 개체로 교체 ← 지금 0개다

[] 식 (7)(8) 의 s_j 를 z_j 로 (제출 시각과 충돌)

[] 식 (8) 의 $avail_r$ 표기를 a_r 로 통일

[] 새 식 (13)(14)(15) 를 §3.6 에 삽입

[] R1 · R4 를 본문 문장으로 명시

[] 본문에서 식 번호를 산문으로 인용 (지금 문단 452·454·456 에서만 부른다.

식 (6)~(12) 는 한 번도 번호로 참조되지 않는다.

Caspian·CarbonFlex 는 알고리즘 블록 밖 산문에서 식 번호를 인용한다)